اول آزمایش کن.
بعد دربارهٔ نسخهٔ جدید تصمیم بگیر.

راهنمای مرحله‌به‌مرحلهٔ آزمایش Cost و History، تغییر پارامتر و کد برین با @phorvexbot؛ از نوشتن درخواست دقیق تا بررسی تست‌ها و مقایسهٔ نسخهٔ آزمایشی با برین مبنا.

فارسی · راست‌به‌چپ | دستورهای قابل کپی | سه مسیر: Study، پارامتر و کد | گزارش HTML + CSV

ویرایش 2026-10-31 · Cost / History پویا

آموزش و ساخت فایل JSON ←

شروع سریعِ دریافت فایل

در بات templates/ بزن، یک نمونه را ویرایش کن و آن را به‌صورت فایل پیوست کن. کپشن validate/ اعتبار ساختار را جدا از آمادگی نسخه، ثبت داده، تعداد snapshot و تاریخچهٔ لازم بررسی می‌کند؛ ارسال فایل با کپشن خالی، آزمایش‌های معتبر را خودکار وارد صف می‌کند. مراحل کامل و مثال‌ها ←

مبنای فعلی: operator-refactored و stage

بنچمارک ثبت‌شده به نسخه و digest دقیق brain در stage، شناسهٔ policy و هش snapshotها متصل است. policy پیش‌فرض phorvex-staging-demo/demo-api-defense است. با policies/ فهرست policyهای مجاز و نصب‌شده را ببینید. policy فعال demo و fixture جداگانهٔ dryRun دو آزمایش متفاوت‌اند؛ بات یکی را جای دیگری نمی‌گذارد. سرویس مجاز همگام‌سازی، تغییر نسخهٔ واقعی را بررسی و تاریخچهٔ قبلی را حفظ می‌کند؛ اعتبارسنجی و فایل ارسالی شما pin یا تنظیمات کلاستر را تغییر نمی‌دهند. دادهٔ قدیمی پروداکشن یا vllm وارد این مسیر نمی‌شود.

دستور param/ مستقیماً نسخهٔ ثبت‌شدهٔ stage را بازپخش می‌کند. bench/ کد واقعی مبنا و کاندید را تست، بیلد و مقایسه می‌کند. V2/V3 ثبت‌شده از موتور همان تصویر اجرایی پس از اثبات بازتولید دقیق، یا موتور آزمایشی با بازبینی مستقل استفاده می‌کند. مجموعهٔ هر درخواست باید کنترل دقیق نسخهٔ مستقر را بازتولید کند؛ مقایسهٔ دو نسخه، بررسی بیلد محلی مبنا را هم لازم دارد. آزمایش ساختگی و مدل مرجع Cost/History با برچسب جداگانه گزارش می‌شوند.

در بنچمارک endpoint پنجرهٔ SLO یعنی W و بازهٔ سابقه یعنی L مستقل‌اند؛ هرکدام باید در تاریخچهٔ پیوستهٔ نگه‌داری‌شده جا بگیرند. مثلاً برای سابقهٔ ۳۰۰ثانیه‌ای از "config": {"lookback": 300} استفاده می‌کند؛ این خروجی معادل تاریخچهٔ ۳۰روزه نیست. کل بازهٔ SLO و تاریخچه باید واقعاً ثبت شده باشد. هزینه و اثر واقعی نامشخص با null می‌آید، نه صفر.

سیاست و تاریخچهٔ دقیقِ درخواست

ابتدا policies/ را ببینید. بنچمارک پارامتر یا کد برای demo-api-defense به snapshot تازه و هم‌نسخه نیاز دارد. مطالعهٔ endpoint همین سیاست فعال، علاوه بر آن به تاریخچهٔ واقعیِ منتسب به جابه‌جایی‌ها نیاز دارد؛ روشن بودن capture این تاریخچه را ایجاد نمی‌کند. کنترل کوتاه روی fixture جداگانهٔ stage-bench-defense جای درخواست demo را نمی‌گیرد.

برای مقدارهای دلخواهِ پشتیبانی‌شده از فایل TXT/Markdown با بخش `[endpoint]` استفاده کنید؛ تغییر عددها به ثبت hash تازه نیاز ندارد. W و L مستقل و بر حسب ثانیه‌اند. کلید نامعتبر یا تکراری رد می‌شود. `sync:false` مانع دانلود در درخواست است، ولی بررسی هویت نسخه را حذف نمی‌کند. اگر min/max را تعیین کنید و eta ندهید، مقدار آن `ln(max/min)/3600` است.

```
[endpoint]
title: V2 versus V3 — requested ten-day endpoint study
policy: phorvex-staging-demo/demo-api-defense
ticks: 64
sync: false
dataset: recorded
study.window: 864000
config.lookback: 86400
study.slo_target: 0.999
config.min: 0.0001
config.max: 1000
config.g_max: 1
config.max_dt: 30
config.delta: 30
```

این نمونه دقیقاً سیاست demo،۶۴ تیک، W=864000، L=86400 و s=0.999 را نگه می‌دارد. نبود تاریخچهٔ پیوستهٔ ده‌روزه، انتساب واقعی جابه‌جایی‌های سیاست فعال یا آمادگی ذخیره‌سازی و پردازش، نتیجهٔ **BLOCKED** با تنظیمات درخواستی و مقدارهای نامعلوم null می‌دهد. سیاست و بازه‌ها خودکار عوض نمی‌شوند. تبدیل دو متن دقیقِ قدیمیِ ده‌روزه و سی‌روزه همراه با رسیدِ تبدیل ثبت می‌شود؛ متن دست‌کاری‌شده همچنان درخواست تغییر کد است.

syntax VALID را جدا از **runtime READY/BLOCKED** بخوانید. اعتبارسنجی نه مدل را صدا می‌زند، نه دانلود می‌کند و نه pin را تغییر می‌دهد. هنگام اجرا بررسی تکرار می‌شود؛ هماهنگی با نسخهٔ جدید فقط توسط سرویس تنظیم‌شدهٔ اپراتور انجام می‌شود. تست‌ها و بازتولید دقیق baseline همچنان شرط نتیجهٔ نهایی هستند.

01 این بات چه کاری می‌کند؟

پارامتر، تغییر محدود کد، یا مقایسهٔ مدل‌های Cost و History.

بات سه مسیر دارد: `/param` مقدارهای پارامتر را با برین مستقر مقایسه می‌کند؛ `/bench` برای یک تغییر محدود کد، patch می‌سازد و آن را روی وضعیت‌های ثبت‌شده بازپخش می‌کند؛ فایل `study` الگوریتم‌های مرجع Cost و History را با بازوها و seedهای مشخص اجرا می‌کند.

> **اول نوع آزمایش را مشخص کن.**
>
> Replay تصمیم‌ها را روی اسنپ‌شات‌های گذشته بررسی می‌کند. Study محاسبات الگوریتم‌های مرجع را روی دادهٔ صریح واقعی یا مصنوعی مقایسه می‌کند. هیچ‌کدام به‌تنهایی نرخ دفاع زنده، هزینهٔ واقعی یا پوشش تأییدشدهٔ تصمیم‌های فرضی را ثابت نمی‌کنند.

بات تغییری را روی برین زنده اعمال نمی‌کند. فایل گزارش، تنظیمات و نوع داده را همراه نتیجه نگه دار. برای شروع Cost و History، راهنمای آزمایش پویا را بخوان.

02

شروع و بررسی آمادگی

یک‌بار دسترسی را بررسی کن؛ قبل از هر آزمایش مهم هم check/ بگیر.

1

وارد گفت‌وگوی خصوصی بات شو

@phorvexbot را باز کن و Start را بزن. اگر هنوز عضو مجاز نیستی، از مالک بات لینک عضویت تازه بگیر؛ لینک را باز کن و Start بزن.

مالک با دستور `/invite` لینک یک‌بارمصرف با اعتبار ۲۴ ساعت می‌سازد.

لینک عمومی بات به‌تنهایی مجوز عضویت ایجاد نمی‌کند. بات به حساب بدون مجوز پاسخ نمی‌دهد.

2

راهنما و وضعیت سرویس‌ها را بگیر

این دو دستور را در دو پیام جداگانه بفرست:

پیام اول · راهنمای خود بات

```
/help
```

پیام دوم · بررسی آمادگی

```
/check
```

برای دریافت آخرین راهنمای انگلیسی به‌صورت دو فایل HTML و PDF، این دستور را جداگانه بفرست:

دریافت راهنمای انگلیسی داخل چت

```
/guide
```

منو و پیام‌های بات انگلیسی‌اند. این فایل آموزش فارسی است؛ دستورها و کلیدهای نمونه را به همان شکل لاتین ارسال کن.

3

پیش‌نیاز مرتبط با مسیرت را بررسی کن

ردیف در check/	تفسیر عملی
Telegram / allowlist / state	بات، حساب مجاز و محل نگهداری کارها باید آماده باشند.
Stage replay runtime / stage brain-replay	نسخهٔ دقیق stage، هویت باینری و tickهای هم‌نسخه با UID یکسان policy باید موجود باشند.
Operator repository	برای `/bench` باید commit دقیق برین مبنا در مخزن سرور موجود باشد.
Free disk / Go / OpenAI	برای تولید کد، اجرای تست و ساخت candidate لازم‌اند؛ آستانهٔ پیش‌فرض دیسک برای `/bench` چهار GiB است.

WARN ممکن است فقط یک مسیر را غیرفعال کند. متن همان ردیف می‌گوید کدام قابلیت آماده نیست. سبز بودن /check موفقیت همهٔ تست‌ها یا نتیجهٔ آزمایش آینده را تضمین نمی‌کند.

پیش‌نیاز Study

آزمایش با trace داخل فایل به مدل، Go، مخزن سورس یا داشبورد نیاز ندارد. برای dataset ثبت‌شده، داده باید جداگانه آماده و نصب شده باشد؛ /check وجود این dataset را بررسی نمی‌کند.

03 انتخاب مسیر مناسب

سه مسیر با ورودی و گزارش متفاوت.

study / JSON

مقایسهٔ Cost و History

بازوها، seedها و تنظیمات را از فایل تعیین کن؛ تولید کد توسط مدل لازم نیست.

مراحل اجرا ←

/param

تغییر مقدار پارامتر موجود

از overrideهای ثبت‌شدهٔ باینری برین مستقر استفاده کن.

مراحل اجرا ←

/bench

تغییر محدود در کد

فایل و تابع مشخص بده؛ مدل patch می‌سازد و تست و replay اجرا می‌شود.

مراحل اجرا ←

مقالهٔ History، فایل Word مدل Cost یا توضیح بلند یک آزمایش، خودکار به ورودی اجرایی تبدیل نمی‌شود. از قالب متناسب با مسیرت استفاده کن.

گزینه‌های /param و /bench

گزینه	معنی	نکته
`policy=phorvex-staging-demo/demo-api-defense`	انتخاب namespace و policy	قالب دقیق `namespace/name` است؛ این policy پیش‌فرض فعلی است.
`ticks=64`	درخواست حداکثر ۶۴ نمونه برای بررسی اولیه	پیش‌فرض فعلی ۵۰۰؛ دامنهٔ قابل درخواست ۱ تا ۵۰۰۰. تعداد واقعی به داده و واجدشرایط بودن نمونه‌ها بستگی دارد.
`nosync`	استفاده از داده و هویت باینریِ موجود روی سرور	بدون این واژه، بات ابتدا sync می‌کند. پس از انتشار جدید، برای تازه‌شدن هویت و داده، آن را حذف کن.

«فعلی» در حالت nosync یعنی آخرین هویت ذخیره‌شده روی سرور.

اگر برین زنده تازه deploy شده باشد، اطلاعات cache ممکن است عقب باشد. commit مبنا را در گزارش بررسی کن. sync، وجود سورس آن commit در مخزن operator را تضمین نمی‌کند؛ کمبود سورس باید جداگانه رفع شود.

04 تغییر پارامتر با /param

برای آزمون یک مقدار قابل override، بدون دست‌زدن به کد.

1 پارامتر و فرضیه را انتخاب کن

نمونه: «اگر سقف هزینهٔ مجاز را تغییر بدهم، تعداد اقدام‌های انتخاب‌شده و هزینهٔ برآوردی چه تغییری می‌کند؟» برای این سؤال، P-FEA-006 یا CHi cost ceiling یک پارامتر مرتبط است.

این سقف با واحد هزینهٔ اختلال نرمال‌شده است، نه دلار. پایین‌تر آوردن آن دروازهٔ هزینه را سخت‌گیرتر و بالاتر بردن آن این دروازه را بازتر می‌کند؛ این رابطه به‌تنهایی کیفیت دفاع را مشخص نمی‌کند.

2 یک مقایسهٔ کوچک اجرا کن

نمونهٔ آزمایشی · حداکثر ۶۴ tick از دادهٔ موجود

```
/param P-FEA-006 1.0 1.2 1.5 policy=phorvex-staging-demo/demo-api-defense ticks=64 nosync
```

بات یک arm کنترل با **مقدار مؤثر ثبت‌شده در همان داده‌ها** اضافه می‌کند و هر عدد تو را هم جداگانه بازپخش می‌کند. کنترل را لازم نیست خودت وارد کنی؛ عدد ۱٫۲ در این مثال هم نباید بدون بررسی گزارش «مقدار فعلی» فرض شود.

از رقم لاتین و نقطهٔ اعشار استفاده کن. در هر دستور ۱ تا ۱۱ مقدار آزمایشی پذیرفته می‌شود. همهٔ اعداد این آموزش نمونه‌اند، نه پیشنهاد برای تنظیم برین زنده.

3 گزارش را بخوان، سپس نمونه را بزرگ‌تر کن

ابتدا وضعیت، تعداد نمونهٔ واقعی و مقایسهٔ کنترل با هر مقدار را بررسی کن. اگر داده و نتیجه قابل تفسیر بود، برای بررسی گسترده‌تر دستور زیر را بفرست. در این نمونه sync هم انجام می‌شود:

نمونهٔ بزرگ‌تر با دریافت دادهٔ تازه

```
/param P-FEA-006 1.0 1.2 1.5 policy=phorvex-staging-demo/demo-api-defense ticks=500
```

نمونه یا حتی نسخهٔ مبنا ممکن است بین دو اجرا عوض شده باشد. نتیجهٔ هر اجرا را با کنترل همان اجرا بخوان؛ عدد دو گزارش متفاوت را بدون بررسی cohort و commit به هم نسبت نده.

چند پارامتر شناخته‌شده و محدودیت آن‌ها

شناسه	معنی و واحد	دامنه و محدودیت
P-FEA-006	سقف هزینهٔ CHi؛ اختلال نرمال‌شده	بزرگ‌تر از صفر تا ۱۰۰۰؛ unset ندارد.
P-FEA-005	فاصلهٔ اقدامات پیشگیرانه؛ ثانیه	عدد صحیح ۱ تا ۶۰۴۸۰۰؛ override هم cadence و هم exposure-pressure scale را تغییر می‌دهد. رویداد معتبر تازه می‌تواند دروازهٔ cadence را دور بزند.
P-OBJ-003	ضریب ثابت disruption aversion یا beta	صفر تا ۱۰ و unset؛ با حالت costPreferences ناسازگار است و ممکن است MODE_CONFLICT بگیری.

مثال دوم ۵۰، ۱۵ و ۳۰ دقیقه برای P-FEA-005، با واحد ثانیه

```
/param P-FEA-005 300 900 1800 policy=phorvex-staging-demo/demo-api-defense ticks=64 nosync
```

وجود یک شناسه در رجیستری به‌معنای امکان override همهٔ پارامترها نیست. باینری replay باید آن را پشتیبانی کند و preflight هم باید تأیید کند. این مثالِ cadence، محدودکنندهٔ یکسان برای همهٔ پاسخ‌های حمله نیست.

05 تغییر کد و فرمول با /bench

از یک فرضیهٔ روشن تا patch، تست و مقایسهٔ سه نسخه.

برای مطالعهٔ Cost و History از JSON استفاده کن.

این فصل مخصوص تغییر کد برین است. مثال افزایش ۲۰٪ ضریب norm، مدل endpoint-second یا History v1.1 را پیاده نمی‌کند. آموزش مسیر Study

ورودی /bench یک توضیح است.

در نسخهٔ فعلی، با این دستور توضیح می‌دهی چه چیزی تغییر کند؛ مدل براساس آن یک diff می‌سازد. انتخاب مستقیم branch یا commit دلخواه، آپلود diff برای اجرای همان patch و دستور /rerun در رابط فعلی وجود ندارد.

1 فقط یک سؤال قابل بررسی تعریف کن

مثال مناسب: «با افزایش ۲۰٪ وزن بخش norm در جریمهٔ هزینه، آیا هزینهٔ برآوردی کاهش می‌یابد؟ هم‌زمان تعداد اقدام‌ها، پوشش پیش‌بینی‌شده و no-op زیر حمله چه می‌شوند؟»

درخواست کلی مثل «برین را بهتر کن» محدوده و معیار مشخصی ندارد. برای آزمایش اول، تغییر را به یک تابع یا یک شرط محدود کن. اگر مسیر یا نام تابع را نمی‌دانی، ابتدا آن را در سورس همان نسخه پیدا کن.

2 این پنج جزء را در درخواستت بنویس

جزء	در مثال فرمول هزینه
محل تغییر	`phorvex/internal/model/economics.go` ، تابع `EconomicsMetadata.CostPenalty`
حالت موردنظر	فقط `independent-cost-preferences-v1`
فرمول یا رفتار جدید	ضریب بخش norm از ۱ به ۱٫۲ برسد؛ بخش flow ثابت بماند.
محدودیت‌ها	حالت‌های legacy و fixed-beta، دروازه‌های SLO، feasibility و بودجه‌ها حفظ شوند.
اندازهٔ تغییر	patch حداقلی؛ بدون تغییرهای نامرتبط یا بازنویسی گسترده.

نام فایل، تابع و شناسه‌ها را دقیق و لاتین بنویس. مثال‌های فنی انگلیسی این صفحه به انتخاب context درست کمک می‌کنند؛ توصیف کاملاً فارسی بدون شناسهٔ کد ممکن است context کافی پیدا نکند. نام و فرمول مثال را با سورس commit مبنای خودت تطبیق بده.

3 فرمول قبل و بعد را شفاف مشخص کن

این مثال برای آموزش تعریف آزمایش است؛ مناسب‌بودن ضریب ۱٫۲ یا عبور آن از تست‌ها از پیش تأیید نشده است.

قبل · فقط در حالت independent-cost-preferences-v1

```
penalty = w_norm * norm + w_flow * flow
```

بعد · افزایش ۲۰٪ سهم norm در تابع جریمه

```
penalty = 1.2 * w_norm * norm + w_flow * flow
```

مثال عددی صرفاً برای فهم فرمول: با `w_norm=0.5` ، `norm=2` ، `w_flow=0.3` و `flow=1` ، جریمه از ۱٫۳ به ۱٫۵ می‌رسد. این افزایش فقط دربارهٔ همین عبارت ریاضی است؛ تعداد اقدام‌ها یا هزینهٔ نهایی برین الزاماً ۲۰٪ تغییر نمی‌کند.

4 دستور کامل را در یک پیام بفرست

نمونهٔ تغییر فرمول · آمادهٔ کپی · آزمایشی

```
/bench policy=phorvex-staging-demo/demo-api-defense ticks=64 nosync In phorvex/internal/model/economics.go, change EconomicsMetadata.CostPenalty in independent-cost-preferences-v1 mode from m.NormPreference.Value*norm + m.FlowPreference.Value*flow to 1.2*m.NormPreference.Value*norm + m.FlowPreference.Value*flow. Preserve legacy and fixed-beta behavior, feasibility gates, SLO checks and budgets. Keep the patch minimal.
```

گزینه‌ها باید قبل از توضیح تغییر باشند.

در /bench ترتیب درست این است: دستور، سپس ...=policy و ...=ticks و در صورت نیاز nosync، سپس متن تغییر. اگر این گزینه‌ها را آخر متن بگذاری، ممکن است به‌عنوان بخشی از درخواست مدل خوانده شوند.

اگر می‌خواهی پیش از آزمایش داده و هویت برین تازه شود، همین دستور را بدون واژهٔ `nosync` بفرست. برای تعداد نمونهٔ متفاوت از سازندهٔ دستور استفاده کن.

5 مراحل اجرا را دنبال کن

بات شناسهٔ job می‌دهد و مراحل را گزارش می‌کند. یک کار در هر لحظه اجرا می‌شود و تا ۵ کار می‌توانند منتظر بمانند. ساخت سرد و اجرای تست ممکن است چندین دقیقه طول بکشد؛ زمان ثابت تضمین نشده است.

پیگیری کار فعلی و صف

```
/status
```

مرحله	بات چه می‌کند؟
انتخاب مبنا و داده	هویت replay و SHA256 را بررسی و tickهای هم‌commit را انتخاب می‌کند؛ دادهٔ انتخاب‌شده داخل همان آزمایش ثابت می‌شود.
تولید patch	مدل تغییر را روی commit مبنا پیشنهاد می‌کند؛ candidate در clone موقت ساخته می‌شود.
تست	تست‌های Go روی هر دو نسخه: `internal/decision` ، `cmd/brain-replay` و packageهای تغییرکرده. این مجموعه، کل CI مخزن نیست.
Build	از مبنا و candidate دو replay می‌سازد؛ نسخهٔ کامپایلر از metadata باینری مستقر خوانده و برای هر دو یکسان انتخاب می‌شود.
Replay و گزارش	سه arm را روی همان tickها بازپخش می‌کند و HTML، CSV، diff و تست‌ها را می‌فرستد.

شکست تست یعنی توقف قبل از بنچمارک عددی.

اگر تستِ مبنا یا candidate رد شود، بات log همان مرحله را می‌فرستد. یک تغییر معتبر در فرمول هم ممکن است با قرارداد عددی موجود ناسازگار باشد؛ تست را نباید صرفاً برای گرفتن نتیجه حذف یا دور زد.

6 چهار خروجی را کنار هم نگه دار

فایل دریافتی	چه چیزی را در آن بررسی کنی؟
report-<job>.html	وضعیت، commitها، پوشش داده، fidelity و شاخص‌های مقایسه
metrics-<job>.csv	اعداد، جامعهٔ نمونه‌ها، فاصله‌ها، دادهٔ موجود/ناموجود و نوع مقایسه
candidate-<job>.diff	اینکه کد تولیدشده دقیقاً همان درخواست تو را اجرا کرده باشد
tests-<job>.json	تست‌های مبنا و candidate، package‌ها، شمارش نتیجه‌ها و نسخهٔ toolchain

نام فایل‌ها شناسهٔ job دارند. diff را پیش از قضاوت دربارهٔ عددها بخوان؛ موفقیت build تضمین نمی‌کند مدل درخواست را درست فهمیده باشد.

هر bench/ از نو و مستقل شروع می‌شود.

دستور دوم تغییرِ دستور اول را ادامه نمی‌دهد؛ candidate تازه‌ای از commit مبنا می‌سازد. ارسال همان توضیح با ticks=500 هم اجرای مجددِ تضمین‌شدهٔ همان patch نیست. diff دو اجرا را مقایسه کن؛ برای اجرای قطعیِ همان candidate به مسیر جداگانهٔ مهندسی نیاز است.

برای اولین بار: یک آزمایش بدون تغییر رفتار

اگر فقط می‌خواهی مسیر تولید patch، تست، build و تحویل گزارش را امتحان کنی، می‌توانی یک کامنت درخواست کنی. این مثال برای بهینه‌سازی نیست؛ انتظار مفهومی آن صفر بودن تغییر تصمیم‌هاست.

بررسی مسیر اجرا · فقط افزودن یک کامنت

```
/bench policy=phorvex-staging-demo/demo-api-defense ticks=64 nosync In phorvex/internal/decision/ablation.go, add one explanatory comment immediately before the existing if base.Ratio > 0 branch, explaining that the positive-ratio check avoids division by zero. Change comments only. Preserve all executable code and behavior.
```

محدودهٔ مجاز candidate: فایل‌های Go زیر phorvex/ ، به‌جز phorvex/tools/ . افزودن فایل _test.go جدید مجاز است؛ تغییر، حذف یا تغییر نام تست‌های موجود مجاز نیست. تغییر Dockerfile، chart، go.mod ، dependency یا حذف فایل در این مسیر پذیرفته نمی‌شود.

06

دریافت فایل و اجرای چند آزمایش

فایل می‌تواند فقط bench، فقط params یا هر دو بخش را داشته باشد.

1

نمونهٔ تکمیل‌شده بگیر

این دستور نمونه‌های فعلی کد/پارامتر، JSON برای Cost و History، و TXT/JSON برای V2/V3 endpoint را می‌فرستد. این فصل قالب‌های کد/پارامتر را توضیح می‌دهد؛ قالب Study جداست.

دریافت فایل‌های نمونه

```
/templates
```

فرمت‌ها: .md ، .txt و Word با پسوند .docx . از Google Docs خروجی Word یا متن بگیر. فرمت قدیمی .doc پشتیبانی نمی‌شود. سقف فایل ۲ MiB و سقف متن استخراج‌شدهٔ این قالب‌ها ۲۵۶ KiB است. فایل JSON مسیر Study می‌تواند تا ۲ MiB باشد.

2

بخش‌های موردنیازت را کامل کن

نمونهٔ زیر یک آزمایش کدِ صرفاً کامنتی و یک مقایسهٔ پارامتر دارد. آن‌ها آزمایش‌های مستقل‌اند و به‌ترتیب اجرا می‌شوند؛ مقدارهای بخش params روی candidate کد اعمال نمی‌شوند. در هر فایل حداکثر پنج آزمایش پذیرفته می‌شود.

حالت اول · هر دو بخش در Markdown یا متن پاراگراف‌های Word

```
# Independent brain experiments

## Bench
policy: phorvex-staging-demo/demo-api-defense
ticks: 64
sync: false
change:
In phorvex/internal/decision/ablation.go, add one explanatory comment immediately
before the existing if base.Ratio > 0 branch, explaining that the positive-ratio
check avoids division by zero. Change comments only. Preserve executable code.

## Params
policy: phorvex-staging-demo/demo-api-defense
ticks: 64
sync: false
P-FEA-006: 1.0, 1.2, 1.5
```

وجود هر دو بخش اجباری نیست.

بخش bench را حذف کنی، فقط پارامترها اجرا می‌شوند. بخش params را حذف کنی، فقط کد بررسی می‌شود. بخش موجودِ خالی یا نامعتبر، کل فایل را رد می‌کند. هر شناسهٔ پارامتر یک آزمایش جدا با کنترل خودش می‌سازد.

حالت دوم · فقط پارامترها؛ این نمونه دو آزمایش می‌سازد

```
# Parameter comparisons

## Params
policy: phorvex-staging-demo/demo-api-defense
ticks: 64
sync: false
P-FEA-006: 1.0 1.2 1.5
P-FEA-005: 300 900 1800
```

حالت سوم · فقط تغییر فرمول؛ ممکن است تست‌های موجود آن را نپذیرند

```
[bench]
policy: phorvex-staging-demo/demo-api-defense
ticks: 64
sync: false
change:
In phorvex/internal/model/economics.go, change EconomicsMetadata.CostPenalty in
independent-cost-preferences-v1 mode from
m.NormPreference.Value*norm + m.FlowPreference.Value*flow to
1.2*m.NormPreference.Value*norm + m.FlowPreference.Value*flow.
Preserve legacy and fixed-beta behavior, feasibility gates, SLO checks and budgets.
Keep the patch minimal.
```

گزینه‌ها را قبل از `change:` قرار بده؛ شرح تغییر می‌تواند چند خط باشد. `sync: false` از دادهٔ ذخیره‌شده استفاده می‌کند و `sync: true` ابتدا sync می‌کند. گزینهٔ حذف‌شده مقدار پیش‌فرض بات را می‌گیرد. برای مقدارهای پارامتر، فاصله یا ویرگول مجاز است.

کلیدها، شناسه‌ها و رقم‌ها را دقیقاً به‌شکل لاتین نمونه نگه دار. فایل‌های متنی را با UTF-8 ذخیره کن. نام فایل و تابع را در شرح تغییر دقیق بنویس.

3 فایل را پیوست کن و حالت اجرا را انتخاب کن

متن نمونه را داخل فایل ذخیره کن و آن را به‌صورت **Document** در چت بات بفرست؛ متن کامل فایل را به‌جای پیوست، در پیام نچسبان. برای Word می‌توانی متن را در پاراگراف‌های معمولی بگذاری و با پسوند `.docx` ذخیره کنی.

روش ارسال	نتیجه
فایل معتبر، کپشن خالی	کل فایل بررسی می‌شود و آزمایش‌ها خودکار به‌صورت یک batch وارد صف می‌شوند.
فایل معتبر، کپشن `/validate`	اعتبار ساختار و آمادگی واقعی داده جداگانه گزارش می‌شوند؛ هیچ همگام‌سازی snapshot یا سورس، تغییر pin، فراخوانی مدل، replay یا صف اجرایی آغاز نمی‌شود.
فایل با یک بخش یا مقدار نامعتبر	کل درخواست رد می‌شود؛ پیام خطا و در صورت امکان شمارهٔ خط می‌گیری. هیچ آزمایشی وارد صف نمی‌شود.
فایل فقط با bench یا فقط با params	فقط همان بخشِ موجود و معتبر اجرا می‌شود.
Word خراب، فایل بزرگ یا فرمت ناسازگار	رد می‌شود؛ از نمونه‌ها یا خروجی تازهٔ Word/TXT استفاده کن.

کپشن را روی همان فایل بگذار.

فرستادن /validate به‌عنوان پیام جدا فقط دستورالعمل را نشان می‌دهد؛ حالت فایلِ بعدی را تغییر نمی‌دهد. برای اعتبارسنجی بدون اجرا، کپشن خود پیوست باید /validate باشد.

اعتبارسنجی ساختار، تضمین کامپایل یا قبولی کد نیست. در اجرای bench، تست‌های Go قبل از بنچمارک عددی اجرا می‌شوند. preflight زمان اجرا هم پشتیبانی باینری، داده و سازگاری پارامتر با حالت اقتصادی را بررسی می‌کند.

4 کارت وضعیت و گزارش‌ها را دنبال کن

هر آزمایش گزارش معمول خودش را می‌فرستد. در پایان، خلاصهٔ JSON و CSV کل batch را هم با شناسهٔ کارها می‌گیری. شکست زمان اجرای یک آزمایش گزارش می‌شود و آزمایش‌های مستقل بعدی ادامه پیدا می‌کنند. `/cancel` درخواست لغو batch جاری را ثبت می‌کند و بقیهٔ آزمایش‌هایش اجرا نمی‌شوند؛ توقف مرحلهٔ جاری ممکن است فوری نباشد.

وقتی منتظر هستی، وضعیت قابل مشاهده است.

برای دانلود و بررسی فایل، مدل، تست، replay، build، ارسال راهنما و نمونه‌ها، یک کارت قابل‌ویرایش نام کار و زمان سپری‌شده را نشان می‌دهد. کارت کارِ در حال اجرا هر هشت ثانیه تازه می‌شود. نوار متحرک نشانهٔ فعالیت است؛ درصد تکمیل یا زمان پایان حدس زده نمی‌شود.

نمونهٔ نمایشی کارت وضعیت؛ خروجی یک کار زنده نیست

```
⁘ BATCH job ab12cd34ef56 — RUNNING
[□□■■■□□□□□] Experiment 2/3 (bench): running Go unit tests: candidate
Elapsed: 3m 10s | Current task: 42s
Activity indicator; no completion-time estimate.
```

با `/status` کار و صف فعلی را ببین. `/runs` آخرین رکوردها را نشان می‌دهد. کارت در پایان وضعیت تمام‌شده، شکست، لغو یا وقفه را ثبت می‌کند.

`PARTIAL` در خلاصهٔ batch یعنی بعضی آزمایش‌ها موفق و بعضی ناموفق بوده‌اند. `PARTIAL` در گزارش یک آزمایش به پوشش ناقص داده اشاره دارد؛ هر دو را جداگانه بخوان.

حالت چهارم · فهرست دستورها داخل فایل TXT

هر خط یک دستور کامل است. برای شرح چندخطی، از قالب بخش‌ها استفاده کن.

command-list.txt

```
/param P-FEA-006 1.0 1.2 1.5 policy=phorvex-staging-demo/demo-api-defense ticks=64 nosync
/bench policy=phorvex-staging-demo/demo-api-defense ticks=64 nosync In phorvex/internal/decision/ablation.go, add one explanatory comment immediately before the existing if base.Ratio > 0 branch, explaining that the positive-ratio check avoids division by zero. Change comments only. Preserve all executable code and behavior.
```

نمونهٔ رد شدن: مقدار `P-FEA-006: 0` خارج از دامنه است، `ticks: 5001` مجاز نیست و بخش `[bench]` بدون change ناقص است. پیش از ارسال دوباره، همهٔ خطاهای گزارش‌شده را اصلاح کن. دستور `/upload` همین روش ارسال را در چت

یادآوری می‌کند.

در Study، یک فایل یک آزمایش با چند بازو می‌سازد. در قالب‌های این فصل، چند بخش آزمایش‌های مستقل‌اند و History روی candidate قبلی Cost سوار نمی‌شود. PARTIAL در Study می‌تواند به‌خاطر دادهٔ ناقص یا fallback انتخاب‌گر باشد.

کپشن را دقیق بنویس.

از /validate با اسلش رو به جلو استفاده کن. کپشن ناشناخته، از جمله \validate، بدون دانلود یا صف‌کردن رد می‌شود. برای اجرا کپشن را خالی بگذار.

07 بنچمارک پویا برای Cost و History

یک فایل JSON؛ چند الگوریتم، seed مشخص و گزارش مشترک، بدون تولید دوبارهٔ کد.

این قابلیت روی بات فعال است.

مسیر study الگوریتم‌های پیاده‌سازی‌شده را با تنظیمات فایل اجرا می‌کند. تغییر seed، پارامترهای انتخاب‌گر و بازوهای مقایسه به درخواست patch از مدل نیاز ندارد. فرمول یا الگوریتم جدیدی که در فهرست پشتیبانی نیست، همچنان به توسعه نیاز دارد.

۱. یکی از سه فایل آماده را انتخاب کن

در بات /templates بزن. برای آشنایی، ابتدا یکی از دو نمونهٔ مصنوعی را اجرا کن. دستور /study راهنمای این مسیر را نشان می‌دهد؛ اجرای آزمایش با پیوست‌کردن فایل انجام می‌شود.

فایل	مقایسه	نوع داده
study-cost-synthetic.json	هزینهٔ legacy و endpoint-v1؛ انتخاب‌گر قطعی یکسان، حداکثر یک حرکت در تیک، سه seed	مصنوعی؛ ۶ ترکیب بازو/seed
study-history-synthetic.json	قطعی، Entropy، تصادفی بدون وزن تاریخچه، و History؛ سه seed	مصنوعی؛ ۱۲ ترکیب بازو/seed
study-history-recorded.json	همان چهار انتخاب‌گر روی ورودی ثبت‌شدهٔ آماده‌شده	واقعی؛ به trace و ledger معتبر نیاز دارد

نمونهٔ مصنوعی، نتیجهٔ عملکرد پروداکشن نیست.

دو فایل synthetic برای یادگیری و بررسی محاسبات آماده‌اند. فایل recorded تا زمان آماده‌شدن دادهٔ لازم، NEEDS_DATA می‌دهد. وجود اسنپ‌شات‌های قدیمی یا PASS بودن /check به معنی وجود دفتر ثبت هزینه و تاریخچهٔ لازم نیست.

۲. تنظیمات را تغییر بده و اعتبارسنجی کن

از سازندهٔ پایین همین بخش فایل بگیر، یا JSON را در ویرایشگر متن تغییر بده. نام کلیدها و شناسه‌ها را لاتین نگه دار. فایل را با کپشن `/validate` بفرست؛ نتیجهٔ ساختار و آمادگی اجرا را جداگانه بخوان؛ سپس همان فایل را بدون کپشن برای اجرا یا دریافت گزارش دقیق BLOCKED بفرست.

تنظیم	کاربرد
arms	۱ تا ۸ بازوی نام‌دار؛ نخستین بازو مرجع مقایسهٔ تصمیم‌هاست. همهٔ بازوها روی ورودی مشترک اجرا می‌شوند.
seeds	۱ تا ۱۶ عدد صحیح متمایز. با ورودی، تنظیمات و پیاده‌سازی یکسان، انتخاب‌های تصادفی بازتولید می‌شوند.
epsilon	حد افت موردانتظار امتیاز، نه افت مجازِ تک‌تک انتخاب‌ها. مقدار صفر فقط انتخاب‌های دارای بهترین امتیاز را مجاز می‌کند.
history.window_seconds	بازهٔ نگاه به گذشته برای شمارنده‌ها؛ تعهد پوشش ۲۴ساعته را تغییر نمی‌دهد.
history.mode	`observed` : مصرف رویدادهای ثبت‌شده. `simulated-starts` : شروع‌های اجرای مصنوعی و صریح، فقط با دادهٔ synthetic.
trace یا **dataset**	دقیقاً یکی را بده: دادهٔ داخل فایل، یا `"dataset":"recorded"` برای دادهٔ آماده‌شدهٔ همان policy روی سرور.

سقف فایل JSON دو MiB، سقف تیک‌ها ۵۰۰۰ و سقف مجموع ردیف‌های کاندید در همهٔ بازوها و seedها صد هزار است. JSON را داخل `[bench]` نگذار. گزینه‌های `ticks` و `sync` متعلق به قالب‌های دیگرند و کلید بالادستی study نیستند.

۳. مدل هزینه و انتخاب‌گر را درست انتخاب کن

گزینه	معنی
cost.kind: provided	مصرف c و rho معتبر از داده؛ مناسب مقایسهٔ انتخاب‌گرها روی هزینهٔ ثابت.
cost.kind: legacy	ترکیب norm و flow داده‌شده با وزن‌های صریح و دروازهٔ chi؛ واحد بی‌بعد است.
cost.kind: endpoint-v1	برآورد زمان و رزرو با واحد endpoint-second، شامل وضعیت نمونهٔ سرد، دادهٔ سانسورشده و قاعدهٔ قرنطینه.
selector.kind	`deterministic` ، `entropy` یا `history` . نمونهٔ History یک بازوی score-only هم با وزن‌های تاریخچهٔ صفر دارد.

برای بررسی اثر History، هزینه و gateها را میان چهار بازو ثابت نگه دار. برای بررسی Cost، انتخاب‌گر و تعداد حرکت را ثابت نگه دار. این مسیر حداکثر یک حرکت در تیک دارد. مقادیر نمونه، پارامترهای آزمایشی‌اند و توصیهٔ تنظیم پروداکشن نیستند.

۴. تاریخچه و دادهٔ واقعی چه شرایطی دارند؟

- `F` فرصت واقعی انتخاب بعد از gateهاست؛ retry یک فرصت تازه نیست.
- `S` شروع اجرای تأییدشده است؛ draw، ارسال ناموفق یا صرفاً انتخاب شدن، S را زیاد نمی‌کند.

- اثر امنیتی فقط با رویداد تأیید معتبر ثبت می‌شود. موفقیت ظاهری اجرای یک حرکت، پوشش تأییدشده ایجاد نمی‌کند.
- تاریخچهٔ هر بازو و seed جداست. در حالت observed، انتخاب‌های فرضی candidate رویداد اجرای واقعی تولید نمی‌کنند.
- تاریخچهٔ ناقص، fallback قطعی با `PICKER_UNAVAILABLE` و گزارش `PARTIAL` می‌دهد. دفتر هزینهٔ غایب به‌صورت بودجهٔ صفرمصرف فرض نمی‌شود.

برای recorded، مسئول داده باید trace زمان‌مرتب با هویت‌ها، gateها، بودجه، نمونه‌های زمان اجرا و رویدادهای معتبر را آماده و وارد کند. تغییر نام فایل synthetic به recorded داده را واقعی نمی‌کند. این ابزار gateها، scheduler یا actuator پروداکشن را دوباره اجرا نمی‌کند؛ verdictهای لازم را از داده می‌گیرد.

نمونهٔ کامل درخواست دادهٔ ثبت‌شده

این فایل از نظر قالب کامل است؛ بدون dataset آماده‌شده، نتیجهٔ درست آن `NEEDS_DATA` خواهد بود.

study-history-recorded.json

```
{
  "schema": "phorvex-bench.study.v1",
  "title": "History v1.1 on normalized recorded evidence",
  "policy": "phorvex-staging-demo/demo-api-defense",
  "dataset": "recorded",
  "seeds": [7, 19, 41],
  "history": {"mode": "observed", "window_seconds": 3600},
  "arms": [
    {"id": "deterministic", "cost": {"kind": "endpoint-v1"}, "selector": {"kind": "deterministic"}},
    {"id": "entropy", "cost": {"kind": "endpoint-v1"}, "selector": {"kind": "entropy", "tau": 1, "epsilon": 0.5}},
    {"id": "score-only", "cost": {"kind": "endpoint-v1"}, "selector": {"kind": "history", "epsilon": 0.5, "score_scale": 1, "eta_pair": 0,
"eta_action": 0, "eta_fact": 0, "kappa_pair": 1, "kappa_action": 1, "q_ref": 1, "x0": 0.7, "a0": 0}},
    {"id": "history", "cost": {"kind": "endpoint-v1"}, "selector": {"kind": "history", "epsilon": 0.5, "score_scale": 1, "eta_pair": 2, "eta_action":
1, "eta_fact": 0, "kappa_pair": 1, "kappa_action": 1, "q_ref": 1, "x0": 0.7, "a0": 0}}
  ]
}
```

هزینهٔ endpoint و لامبدای تطبیقی در V2/V3

برای این مقایسه، فایل `endpoint-v3-synthetic.json` را برای اجرای ۶۴ تیک با دادهٔ صریحاً ساختگی و فایل `endpoint-v3-recorded.json` را برای درخواست دادهٔ ثبت‌شدهٔ معتبر استفاده کنید. بازوهای ساختگی در هستهٔ مرجع مشترک Go اجرا می‌شوند. بازوهای ثبت‌شده از موتور بازبینی‌شدهٔ brain و snapshot واقعی stage استفاده می‌کنند و به بازتولید دقیق مبنا نیاز دارند. درخواست بنچمارک، کاندید را دیپلوی نمی‌کند.

JSON را به صورت فایل بفرستید. کپشن `/validate` ساختار و آمادگی نسخه و داده را بدون اجرا بررسی می‌کند؛ ارسال بدون کپشن، اجرا یا گزارش پیش‌نیازهای مفقود را آغاز می‌کند. خروجی شامل HTML، جدول CSV کاندیدها، JSON کامل و ZIP شواهد است. وضعیت V3 بر اساس زمان ثبت‌شده پیش می‌رود و هزینهٔ واقعی از انتخاب‌های فرضی ساخته نمی‌شود.

BLOCKED یعنی دادهٔ ضروری موجود نیست.

snapshotهای قدیمی، تاریخچهٔ افت endpoint منتسب به حرکت و خروجی کامل کاندیدها برای قواعد هزینهٔ جدید را ندارند. بات پیش از مصرف مدل و بیلد، کمبودها را مشخص می‌کند؛ تاریخچهٔ ثبت‌نشده قابل بازسازی نیست. COMPLETE برای دادهٔ ساختگی، صحت محاسبهٔ مرجع را نشان می‌دهد، نه آماده‌بودن دادهٔ پروداکشن. فایل TXT اصلی و بررسی‌شدهٔ V3 هم به همین بررسی هدایت می‌شود؛ تغییر دلخواه متن، آن را وارد مسیر بنچمارک کد می‌کند.

در فایل درخواست، study.slo_target ، study.window (پنجرهٔ W بر حسب ثانیه)، study.worth_threshold و config.lookback (پنجرهٔ L بر حسب ثانیه) قابل تغییرند. تغییر روی یک کپی جداگانه با accepted=false و study_assumption=true اعمال می‌شود؛ gate، facts‌ها، شناسه‌ها، بازه‌های حرکت، نمونه‌های مدت و اعتبار ledger تغییر نمی‌کنند. پیش از هر تیک انتخاب‌شده باید کل W و L واقعاً مشاهده شده باشد. افزایش پنجره به تأیید نگهداری کامل تاریخچه در خروجی نرمال‌سازی هم نیاز دارد. در صورت کمبود تاریخچه، پیش از اجرای بازوها BLOCKED همراه با مدت لازم و مدت موجود برمی‌گردد.

قالب کوتاه برای phorvex-staging-demo/stage-bench-defense با W=600، L=600 و s=0.9 است. قالب اصلی W=2,592,000 (۳۰ روز)، L=86,400 (یک روز) و s=0.999 را می‌خواهد و به تاریخچهٔ پیوستهٔ نگه‌داری‌شده، ظرفیت کافی ذخیره‌سازی و تأیید امکان پردازش در زمان مجاز نیاز دارد. قابلیت اجرای ۳۰روزه هنوز اعتبارسنجی نشده است. موفقیت مطالعهٔ کوتاه به معنی تکمیل درخواست ۳۰روزه نیست. فایل dataset-manifest.json منشأ اصلی نرمال‌سازی و study-settings.json تنظیمات درخواستی و مؤثر را نگه می‌دارند.

دادهٔ تازهٔ stage. همگام‌سازی و تازه‌سازی دوره‌ای stage، داده‌های تازه با منبع، تصویر اجرایی و UID مشخص را جمع می‌کنند. هر درخواست ثبت‌شده یک مجموعهٔ معتبر را برای خودش ثابت نگه می‌دارد و درخواست بعدی می‌تواند از مجموعهٔ تازه‌تری استفاده کند. اگر تاریخچه کافی نباشد یا منبع تغییر کرده باشد، آخرین manifest معتبر حفظ می‌شود و علت وضعیت فعلی در mathlab/endpoint-refresh/NAMESPACE/POLICY.status.json ثبت می‌شود. بازهٔ زمانی و منبع واقعی در نتیجه باقی می‌مانند؛ مجموعهٔ قدیمی به نام دادهٔ تازه گزارش نمی‌شود. درخواست W=۳۰ روز و L=یک روز به تمام این تاریخچهٔ پیوسته و تأیید آمادگی ذخیره‌سازی و پردازش نیاز دارد؛ قابلیت اجرای ۳۰روزه هنوز اعتبارسنجی نشده است.

مقدار budget.acting_admission = BLOCKED: offline experiment طبیعی است: بازپخش مجوز اجرای فیزیکی نمی‌دهد. برای تشخیص موفقیت مقایسه، state اصلی نتیجه، comparable_ticks و دلایل کمبود داده را بخوانید؛ این برچسب به‌تنهایی به معنی شکست بنچمارک نیست. این فایل‌ها در /templates هم هستند. نوع evidence ، تعداد comparable_ticks ، نتیجهٔ هر دو بازو، جزئیات لامبدا و علت کمبود داده را بررسی کنید. هزینهٔ واقعی و اثر امنیتی نامشخص با null نمایش داده می‌شوند.

08 سازندهٔ دستور قابل کپی

گزینه‌ها را انتخاب کن، سپس خروجی را در تلگرام بفرست.

این ابزار فقط متن دستور را داخل همین صفحه می‌سازد؛ به بات یا سرور وصل نمی‌شود. مقدارهای نمونه برای آموزش‌اند و پیشنهاد تنظیم production نیستند.

این فرم ترتیب و بعضی ورودی‌ها را بررسی می‌کند. وجود فایل در commit مبنا، درستی تغییر مدل، پشتیبانی باینری و موفقیت preflight فقط در خود بات مشخص می‌شود.

09 گزارش را به چه ترتیبی بخوانم؟

اول اعتبار مقایسه، بعد تفاوت شاخص‌ها، سپس تفسیر فرضیه.

گزارش Study: ابتدا نوع داده و وضعیت را بخوان

وضعیت	معنی
COMPLETE	محاسبات مطالعه با ورودی کافی انجام شده؛ تأیید کیفیت الگوریتم یا اثر واقعی نیست.
PARTIAL	برخی ورودی‌ها ناقص‌اند یا انتخاب‌گر به حالت قطعی برگشته؛ علت و تعداد fallback را بخوان.
NEEDS_DATA	dataset آماده نیست یا دادهٔ ضروری برای اجرای معتبر وجود ندارد؛ داده را تأمین کن.

وضعیت	معنی
CANCELLED	درخواست لغو اجرا شده؛ شواهد جزئی ممکن است ذخیره شده باشند.

فایل / شاخص	کاربرد
study-<job>-report.html	خلاصهٔ بازوها و seedها، نوع داده، کمبودها و مقایسهٔ تصمیم‌ها.
study-<job>-metrics.csv	تیک‌های قابل استفاده، fallback، تعداد انتخاب و افت موردانتظار امتیاز.
study-<job>-result.json	وضعیت قابل‌پردازش، علت کمبود داده و معیارهای موجود یا ناموجود.
study-<job>-evidence.zip	ورودی و manifest، جدول candidates.csv، خروجی ticks.csv و احتمال‌های هر انتخاب.

افزایش درصد تغییر تصمیم، به‌معنی بهتر شدن نیست. هزینهٔ بی‌بعد legacy را با endpoint-second به‌صورت درصد کاهش مقایسه نکن. null یا unavailable صفر نیست. verified_coverage و realized_cost برای اثر فرضی candidate مشاهده نشده‌اند؛ از انتخاب شدن یا مجموع رزروها، پوشش واقعی یا خرج واقعی نتیجه نگیر.

ادامهٔ این فصل: گزارش replay برای /bench و /param

1 وضعیت و تعداد داده را بررسی کن

عبارت گزارش	معنی و اقدام تو
COMPLETE	اجرای مقایسه با پوشش مورد انتظار تمام شده؛ این برچسب به‌تنهایی نشانهٔ بهبود نیست.
PARTIAL	بخشی از داده واجدشرایط یا قابل مقایسه نبوده است. مخرج‌ها و علت حذف‌ها را بخوان؛ آن را با گزارش کامل هم‌ارز فرض نکن.
FAILED / INVALID	نتیجه برای پذیرش candidate معتبر نیست؛ ابتدا مشکل گزارش‌شده را رفع کن.
64/64 یا مشابه	تعداد واقعی را برای کنترل و هر arm جداگانه بخوان؛ درخواست ۶۴ یا ۵۰۰ لزوماً همین تعداد نمونهٔ معتبر تولید نمی‌کند.

2 در /bench این سه arm را بشناس

control

باینری برین مستقر

فقط stickی وارد کنترل معتبر می‌شود که خروجی ثبت‌شده را دقیقاً بازتولید کند.

base-local

کد مبنا، ساخته‌شده روی سرور

برای اعتبار مقایسه، fidelity باید `EXACT` و تغییر تصمیم آن نسبت به کنترل صفر باشد.

candidate

کد تولیدشده برای درخواست تو

نسخهٔ متفاوتی از برین است؛ نتیجهٔ آن `OBSERVATIONAL` گزارش می‌شود.

اگر base-local برابر DIVERGED بود، نتیجهٔ candidate را مبنای تصمیم قرار نده.

ابتدا علت اختلاف مبنا با باینری مستقر باید روشن شود. EXACT در این گزارش به بازتولید تصمیم‌ها روی tickهای بررسی‌شده اشاره دارد؛ به‌تنهایی تضمین یکسانی همهٔ بایت‌های دو باینری نیست.

در `/param` انتظار معمول `MATCHED_REPLAY` است. عبارت `OBSERVATIONAL` در `/bench` به‌دلیل مقایسهٔ دو revision متفاوت طبیعی است.

3 شاخص‌ها را با سؤال آزمایش کنار هم بگذار

شناسه	چه چیزی را می‌سنجد؟	چطور بخوانی؟
M-P21	سهم tickهایی که مجموعهٔ اقدام انتخاب‌شده با کنترل فرق دارد	شدت تغییر رفتار؛ جهت خوب/بد ندارد و امتیاز بهبود نیست.
M-P26	تعداد اقدام انتخاب‌شده به‌ازای tick	کاهش آن می‌تواند اقدام اضافه را کم کند یا پاسخ لازم را از دست بدهد.
M-P28	هزینهٔ اختلال برآوردِ اقدام‌های انتخاب‌شده	خروجی مدل هزینه است؛ اختلال واقعی یا مبلغ پول اندازه‌گیری‌شده نیست.
M-P19	نسبت asset-tickهای دارای شواهد حمله که اقدامی انتخاب نکرده‌اند	بالا رفتن آن نیازمند بررسی علت no-op و رفتارهای ازدست‌رفته است.
M-P22	سهم فرصت‌های دارای شواهد کامل که اقدام مجاز با gain مثبت دارند	فرصت مدل‌شده را توصیف می‌کند؛ موفقیت دفاع زنده نیست.
M-P25	پوشش پیش‌بینی‌شده با وزن‌های مشترک کنترل	برای مقایسهٔ برآورد پوشش مفید است؛ نتیجهٔ امنیتی مشاهده‌شده نیست.

جامعه و مخرج همهٔ شاخص‌ها یکسان نیست. مقدار `n/a` یعنی داده یا سنجش در دسترس نیست، نه صفر. فاصلهٔ گزارش‌شده و تعداد نمونه‌ها را هم کنار مقدار اصلی بخوان.

4 «تغییر رفتار» را از «بهبود» جدا بخوان

مثال کاملاً فرضی برای آموزش خواندن نتیجه؛ این اعداد از اجرای واقعی بات نیامده‌اند.

شاخص	کنترل	Candidate	تفسیر محدود به همین شاخص
تغییر تصمیم M-P21	۰٪	۲۵٪	از هر چهار tick مقایسه‌شده، یکی تصمیم متفاوت دارد.
اقدام در هر tick M-P26	۰٫۸	۰٫۶	میانگین اقدام‌ها ۲۵٪ کمتر است.
هزینهٔ برآوردی M-P28	۰٫۴۰	۰٫۳۲	میانگین هزینهٔ مدل‌شده ۲۰٪ کمتر است.
No-op زیر حمله M-P19	۱۰٪	۱۵٪	۵ واحد درصد بیشتر؛ نیازمند بررسی اثر نامطلوب احتمالی.

از این مثال نمی‌توان گفت «برین ۲۵٪ بهتر شد».

هم هزینه کمتر شده و هم no-op در وضعیت دارای شواهد حمله بیشتر شده است. معیارها باید در کنار فرضیه و محدودیت‌های ایمنی تفسیر شوند؛ قبولی برای انتشار از یک عدد منفرد به دست نمی‌آید.

در CSV، ردیف ARM_LEVEL مقدار خود arm است و PAIRED_DIFF_VS_CONTROL اختلاف روی نمونه‌های جفت‌شده؛ آن‌ها را با هم اشتباه نگیر. هدفِ M-P18 هم با قیمت‌گذاری هر arm محاسبه می‌شود و با تغییر فرمول هزینه، مقایسهٔ خام آن می‌تواند گمراه‌کننده باشد.

یک شاهد واقعی از کارکرد مسیر؛ نه شاهد بهینه‌سازی

در بررسی ثبت‌شدهٔ 2026-10-28، job با شناسهٔ 0deacfd4c571 روی مبنای 0058a3edd2eb با candidate صرفاً کامنتی تکمیل شد: ۶۴/۶۴ tick، مبنای EXACT، و برای هر نسخه ۳۹۸ رویداد تست PASS، دو SKIP و صفر شکست. تغییر تصمیم صفر بود. این یک شاهد تاریخی از اجرای مسیر است؛ وضعیت فعلی را با /check و گزارش تازه بسنج.

10 اگر خطا گرفتم چه کار کنم؟

شناسهٔ job، نام مرحله و متن خطا را کنار هم نگه دار.

نشانه یا پیام	معنی محتمل	گام بعدی
NEEDS_DATA	دادهٔ لازم Study موجود نیست.	result.json و data_gaps را بخوان؛ dataset واقعی را آماده کن. اجرای دوبارهٔ همان فایل دادهٔ غایب را ایجاد نمی‌کند.
PICKER_UNAVAILABLE	تاریخچه یا ورودی fact لازم ناقص است.	پنجرهٔ تاریخچه و رویدادها را بررسی کن؛ انتخاب fallback را نتیجهٔ کامل History معرفی نکن.

نشانه یا پیام	معنی محتمل	گام بعدی
`Study validation / unknown fields`	کلید، نوع داده، seed یا محدودیت ورودی اشتباه است.	از نمونهٔ JSON شروع کن؛ seed تکراری، NaN و کلیدهای ticks/sync در study پذیرفته نمی‌شوند.
`no in-scope diff ... model declined`	مدل برای /bench تغییر کد مجاز تولید نکرده؛ هنوز benchmark اجرا نشده.	فایل plan-<job>.json را بررسی کن. برای آزمایش‌های پشتیبانی‌شدهٔ JSON، Cost/History مسیر Study را بده؛ برای تغییر کد، درخواست را محدود و دقیق کن.
File rejected / خطای خط یا بخش	قالب، کلید، مقدار یا بخش خالی نامعتبر است.	از templates/ شروع کن، همهٔ خطاها را اصلاح و با کپشن /validate دوباره ارسال کن. کلیدهای نمونه را ترجمه نکن.
PASS برای فایل، سپس شکست آزمایش	ساختار معتبر بوده، ولی preflight زمان اجرا، تست یا build رد شده است.	گزارش و log همان job را بررسی کن. اعتبارسنجی فایل تضمین تست یا پوشش داده نیست.
PARTIAL در خلاصهٔ batch	بعضی آزمایش‌ها موفق و بعضی ناموفق بوده‌اند.	JSON/CSV batch را برای شناسه و وضعیت هر آزمایش بخوان. این وضعیت با PARTIAL مربوط به پوشش دادهٔ یک گزارش فرق دارد.
بات هیچ پاسخی نمی‌دهد	حساب مجاز نیست، لینک دعوت مصرف/منقضی شده یا سرویس در دسترس نیست.	در گفت‌وگوی خصوصی تست کن؛ مالک لینک تازه بدهد. اگر عضو هستی و همچنان پاسخی نیست، وضعیت سرویس باید بررسی شود.
`BOT_REPO does not contain the deployed commit`	باینری جدید موجود است، ولی سورس دقیق آن در clone سرور نیست.	مسئول سرور همان commit را در BOT_REPO fetch کند؛ سپس `/check` و آزمایش تازه. تغییر tag لازم نیست.
`no recorded ticks` **یا ناسازگاری revision**	دادهٔ هم‌نسخه برای policy انتخاب‌شده موجود نیست.	policy را بررسی و بدون `nosync` اجرا کن. اگر ثبت داده فعال نیست، باید capture همان policy بررسی شود.
دیسک کمتر از آستانه	Guard شروع `/bench` مانع ادامه شده است.	روی سرور فضا آزاد شود؛ بعد `/check` . صرفاً پایین آوردن guard مشکل کمبود فضا را رفع نمی‌کند.
`MODE_CONFLICT`	پارامتر در economics mode فعلی معتبر نیست؛ مثل beta در costPreferences.	پارامتر سازگار انتخاب کن؛ برای تحمیل آزمایش، حالت برین زنده را تغییر نده.
مقدار نامعتبر / پارامتر پشتیبانی‌نشده	شناسه، دامنه، واحد یا قابلیت override درست نیست.	از مقدارهای لاتین، واحد درست و پارامتر مورد پشتیبانی استفاده کن؛ پیام preflight را بخوان.
`LLM step failed` **یا patch ردشده**	پاسخ مدل، context یا دامنهٔ فایل‌ها مشکل دارد.	نام دقیق فایل و تابع و یک تغییر کوچک‌تر بده؛ مسیرهای خارج از scope را حذف کن. خطای احراز هویت/سهمیه نیازمند بررسی تنظیم سرویس است.
`base-local Go tests FAILED`	حتی مبنا روی سرور تست را پاس نکرده است.	log مبنا بررسی شود؛ می‌تواند مشکل کد، ابزار یا فشار منابع باشد. این نتیجه را شکست فرمول candidate تلقی نکن.
`candidate Go tests FAILED`	candidate یکی از تست‌ها را نقض کرده است.	log را بخوان. ممکن است تغییر اشتباه باشد یا قرارداد مورد انتظار نیاز به بازبینی مستقلِ کد و تست داشته باشد. تست‌ها را برای گرفتن خروجی دور نزن.
`build failed`	خطای کامپایل، toolchain، ماژول یا منابع	مرحلهٔ خطا و log ساخت بررسی شود؛ هنوز نتیجهٔ معتبر بنچمارک تولید نشده است.
`DIVERGED`	مبنای ساخته‌شده تصمیم‌های کنترل را بازتولید نمی‌کند.	مقایسهٔ candidate را متوقف کن و اول برابری مبنا را رفع کن.

نشانه یا پیام	معنی محتمل	گام بعدی
`n/a` **یا پوشش ناقص**	شواهد کافی، مخرج معتبر یا نمونهٔ قابل بازپخش موجود نیست.	data_status، reason و تعدادها را بخوان؛ `n/a` را صفر ثبت نکن.

برای پیگیری خطا این‌ها کافی‌اند:

شناسهٔ job، متن دستور ارسالی، نام مرحلهٔ شکست و فایل log یا report همان job. چند آزمایش سنگینِ مشابه را پشت سر هم وارد صف نکن؛ ابتدا نتیجهٔ خطای اول را روشن کن.

11 چک‌لیست قبل از تصمیم دربارهٔ تغییر

تکمیل این موارد کمک به بازبینی است؛ مجوز خودکار انتشار ایجاد نمی‌کند.

- [ ] مسیر آزمایش و سؤال روشن است: study، param یا bench؛ واحد معیارها را می‌دانم.
- [ ] نوع دادهٔ synthetic یا recorded، هویت منبع و کامل‌بودن ورودی‌ها را بررسی کرده‌ام.
- [ ] در Study، بازوها، seedها و ورودی مشترک را نگه داشته‌ام و مرجع مقایسه روشن است.
- [ ] در History، فرصت، شروع اجرای تأییدشده و تأیید اثر را جدا می‌دانم و fallback را بررسی کرده‌ام.
- [ ] برای bench، diff/ را خوانده‌ام؛ تست‌ها و مبنای EXACT را بررسی کرده‌ام.
- [ ] وضعیت نهایی، تیک‌های قابل استفاده، دادهٔ ناقص و علت‌های حذف را بررسی کرده‌ام.
- [ ] تغییر تصمیم را با بهبود، هزینهٔ تخمینی را با خرج واقعی، و افت میانگین امتیاز را با افت هر draw یکی نمی‌دانم.
- [ ] فایل ورودی، گزارش‌ها و شواهد را نگه داشته‌ام؛ ادعای نتیجهٔ واقعی به داده یا ارزیابی واقعی نیاز دارد.

۰ از ۸ مورد بررسی شده؛ انتخاب‌ها با بستن صفحه ذخیره نمی‌شوند.

یک یادداشت نتیجهٔ خوب این شکلی است:

«روی commit مبنای مشخص و N نمونهٔ معتبر، candidate با diff مشخص تست‌ها را پاس کرد. مبنا EXACT بود. شاخص‌های X و Y این مقدار تغییر کردند؛ شاخص Z ناموجود/نامطلوب بود. این نتیجه صرفاً replay است و برای پذیرش نهایی، بازبینی و ارزیابی زنده لازم است.»

12 مرجع سریع و سؤال‌های رایج

دستورهای کنترل اجرا را در پیام‌های جداگانه بفرست.

دستور	کاربرد
/study	راهنمای JSON مطالعه؛ فایل را برای اجرا پیوست کن.
/start یا /help	نمایش راهنمای بات برای حساب مجاز
/check	بررسی پیش‌نیازها و دسترسی سرویس‌ها
/guide	دریافت راهنمای انگلیسی HTML و PDF در همان چت
/upload	دستورالعمل ارسال فایل Markdown، TXT و Word
/validate	کپشن فایل برای اعتبارسنجی بدون اجرا؛ پیام جدا حالت فایل بعدی را تغییر نمی‌دهد.
/templates	دریافت نمونه‌های فعلی کد/پارامتر، JSON برای Cost/History و TXT/JSON برای endpoint
/bench [options] <description>	ساخت و آزمون candidate برای تغییر کد؛ گزینه‌ها قبل از شرح تغییر
/param <ID> <values...> [options]	مقایسهٔ مقدارهای پارامتر با کنترل ثبت‌شده
/status	مرحلهٔ کار در حال اجرا و صف فعلی
/runs	آخرین پنج رکورد از تاریخچهٔ اجراها
/cancel	درخواست لغو کار جاری پس از مرحلهٔ فعلی؛ توقف ممکن است فوری نباشد.
/invite	فقط مالک در چت خصوصی: لینک یک‌بارمصرفِ عضویت با اعتبار ۲۴ ساعت

دیدن اجراهای اخیر	لغو کار جاری؛ بعد از مرحلهٔ فعلی
/runs	/cancel

▼ آیا هر فایل Word یا توضیح طولانی خودکار بنچمارک می‌شود؟

خیر. برای کد/پارامتر از بخش‌های معتبر، برای Cost/History از JSON با schema مشخص و برای V2/V3 پشتیبانی‌شده از بخش [endpoint] در TXT/Markdown یا endpoint JSON استفاده کن. متن آزادِ ویرایش‌شده درخواست تغییر کد باقی می‌ماند؛ الگوریتم جدید به پیاده‌سازی نیاز دارد.

▼ آیا نتیجهٔ Study همان اجرای برین پروداکشن است؟

Study اجرای الگوریتم مرجع روی ورودی صریح است. گزارش mode=REFERENCE_STUDY و نوع داده را مشخص می‌کند؛ برای سنجش اثر زنده به شاهد مناسب نیاز است.

▼ چطور یک Study را تکرار کنم؟

همان فایل و همان داده، seed و نسخهٔ پیاده‌سازی را استفاده کن. dataset روی سرور ممکن است عوض شود؛ ورودی تثبیت‌شدهٔ resolved-request.json داخل evidence.zip را نگه دار.

▼ آیا /validate موفقیت Study را تضمین می‌کند؟

خیر. اعتبار ساختار و آمادگی فعلی اجرا دو نتیجهٔ جدا هستند. هویت stage، ثبت داده، تیک‌های معتبر و تاریخچهٔ endpoint پیش از اجرا بررسی می‌شوند. هنگام اجرا هویت و شواهد دوباره کنترل و replay دقیق و تست‌های لازم اجرا می‌شوند. مدل مرجع Cost/History هنوز دفتر کامل مخصوص خودش را هنگام اجرا بررسی می‌کند و کمبود داده می‌تواند NEEDS_DATA یا PARTIAL بدهد. READY تضمین قبولی تغییر کد تولیدشده نیست.

▼ آیا validate/ کد را کامپایل یا تست‌های Go را اجرا می‌کند؟

خیر. کپشن validate/ اعتبار ساختار را جدا از نسخهٔ واقعی، ثبت داده، تعداد و تازگی snapshot و تاریخچهٔ لازم گزارش می‌کند؛ هیچ همگام‌سازی snapshot یا سورس، تغییر pin، فراخوانی مدل یا صف اجرایی انجام نمی‌شود. برای اجرا، فایل را با کپشن خالی بفرست. تست Go و replay مربوط به مسیرهای کد/پارامترند؛ Study بررسی داده و محاسبات خودش را دارد.

▼ اگر bench و params در یک فایل باشند، مقدارها روی کد جدید اعمال می‌شوند؟

هر بخش آزمایش مستقل با کنترل و گزارش خودش می‌سازد و به‌ترتیب فایل اجرا می‌شود. مقدارهای params با باینری مبنا مقایسه می‌شوند. برای بررسی ترکیب یک candidate ثابت با overrideهای پارامتر به مسیر مهندسی جداگانه نیاز است.

▼ چرا نوار وضعیت درصد تکمیل نشان نمی‌دهد؟

مدت انتظار مدل، تست، build و replay از پیش معلوم نیست. کارت با نوار متحرک، مرحلهٔ واقعی و زمان سپری‌شده را نشان می‌دهد و هنگام کار هر هشت ثانیه تازه می‌شود. وقتی کار تمام یا لغو شود، شکست بخورد یا بات ری‌استارت شود، وضعیت نهایی روی کارت ثبت می‌شود.

▼ آیا می‌توانم بعد از نتیجه بگویم «همان قبلی را با ۵۰۰ tick اجرا کن»؟

در نسخهٔ فعلی خیر. هر bench/ یک درخواست مستقل است و مدل دوباره patch می‌سازد. حتی توضیح یکسان تضمین نمی‌کند diff یکسان بگیری. برای مقایسهٔ دو اجرا، خود diff، مبنا و داده را بررسی کن؛ rerun/ وجود ندارد.

▼ آیا می‌توانم فقط با یک توضیح فارسی تغییر را درخواست کنم؟

می‌توانی توضیح فارسی بنویسی، ولی مسیر فایل، نام تابع و شناسه‌های انگلیسی را دقیق اضافه کن. بازیابی context فعلی بر شناسه‌های لاتین تکیه دارد؛ درخواست کاملاً فارسی ممکن است context مفید پیدا نکند. نمونه‌های فنی انگلیسی همین صفحه انتخاب مطمئن‌تری برای تعریف درخواست‌اند.

▼ اگر فرمول عوض شود و تست قدیمی شکست بخورد، بات خودش تست را اصلاح می‌کند؟

خیر. تست‌های موجود تغییرناپذیرند. بات می‌تواند برای helper تازه، فایل تست جدید اضافه کند؛ تست قدیمی را برای پاس‌شدن تغییر بازنویسی نمی‌کند. باید مشخص شود کد جدید غلط است یا قرارداد محصول عمداً قرار است تغییر کند. تغییر قرارداد و تست نیاز به بازبینی جداگانه دارد؛ بات نتیجهٔ شکست را با log گزارش می‌کند و قبل از بنچمارک متوقف می‌شود.

▼ تست PASS شد؛ یعنی کل برین و انتشار جدید تأیید است؟

خیر. تست‌های این مسیر هدفمندند و کل CI، همهٔ packageها یا تست‌های live را پوشش نمی‌دهند. diff، کیفیت شواهد، معیارهای مرتبط و اثرهای زنده هم باید متناسب با تغییر بازبینی شوند.

▼ تغییر تصمیم صفر است؛ یعنی فرمول هیچ اثری ندارد؟

فقط می‌توان گفت روی tickهای معتبرِ این نمونه، مجموعهٔ اقدام انتخاب‌شده فرق نکرده است. ممکن است بخش تغییرکرده فعال نبوده، دروازهٔ دیگری تعیین‌کننده بوده، رتبه‌بندی عوض نشده یا نمونه مناسب نباشد. اول diff و context آزمایش را بررسی کن.

▼ چطور فایل گزارش HTML را باز کنم؟

فایل report را از تلگرام دانلود و در مرورگر باز کن. CSV برای بررسی عددها در Excel یا ابزار داده است؛ diff را با ویرایشگر متن بخوان. این راهنما هم یک فایل مستقل است و برای مطالعهٔ متن به اینترنت نیاز ندارد.

▼ اگر چند بار مقدار ۱٫۲ را تست کنم نتیجه همیشه یکی است؟

داده، هویت مبنا و نمونهٔ انتخاب‌شده ممکن است بین اجراها عوض شوند؛ sync این احتمال را بیشتر می‌کند. در bench/ خود patch هم دوباره تولید می‌شود. نتیجه‌ها را با شناسهٔ diff، commit، job و cohort همان اجرا مقایسه کن.

▼ بعد از پیدا شدن candidate خوب چه می‌شود؟

بات تغییر را منتشر نمی‌کند. diff و شواهد را وارد مسیر بازبینی کد، CI و ارزیابی زندهٔ متناسب با ریسک تغییر کن. مزیت این مرحله، داشتن شواهد مشخص برای آن تصمیم است؛ نتیجهٔ replay به‌تنهایی مجوز اعمال روی محیط زنده نیست.

مبنای این آموزش: پیاده‌سازی `benchmark/bot/service.py`، `planner.py`، `candidate.py`، `replay_build.py`، `code_experiment.py`، `report.py`، `file_requests.py` و `progress.py`، `study.py`، `study_math.py` و `study_schema.py`؛ رجیستری پارامترها و شاهدهای `benchmark/bot/VALIDATION.md`.

این سند در 2026-10-31 تهیه شده است. متن help/ و نتیجهٔ واقعی check/ را برای تغییرهای بعدی سرویس بررسی کن. این فایل هیچ رمز، توکن یا لینک خصوصی عضویت ندارد.